

Proving `exp100.c` with VST

The plan, cut into tasks for separate agents

`code/APaul/exp100/exp100.c` computes an enclosure of $\exp(x)$ to about 156 bits on limbs of 32 bits, and the filter `maybe_hard_bits`. The goal is one Rocq theorem: **for every 64-bit word `xb`, if `maybe_hard_bits(xb)` returns 0 then `x` is not hard**, with `x` the double of bits `xb` and `hard` the predicate of `code/exptablekl/ExpHard.v`. This note fixes the statements, the files and the order of the work, following steps 1–6 of `exp100/README`. Numbers are marked **measured** (a run, this note) or **guessed**.

1 What was measured

All in the opam switch `vst` (Rocq 9.0, CompCert 3.17, VST 2.16), in a scratch directory, then copied to `code/APaul/vst/exp100/` (make there, 24 s).

- `clightgen -normalize exp100.c` runs in 0.07 s and gives a file of 110 KB (2277 lines); `coqc` takes 2.4 s on it. The union wrappers `exp_encl` and `maybe_hard` are in it too (they are simply left out of the proof).
- The globals come out as read-only `gvars` with their init data: `v_T : tarray (tarray tulong 6) 64`, `v_C : tarray (tarray tulong 6) 17`, `v_LN2`, `v_RMAX : tarray tulong 6`, `v_INV : tulong`, each a list of `Init_int64 (Int64.repr z)`.
- A file `Spec.v` with `num_zero_spec`, `num_add_spec` and the shape of `exp_core_spec` (with `GLOBALS (gv)` and the 2D array `T` at `gv _T`) type-checks.
- `body_num_zero` (one counted loop, `data_at_` to zeros): 13 lines of proof, 4.3 s.
- `body_num_add` (the carry loop, against `valZ` on limbs of 32 bits): 95 lines with its two word lemmas (`x & 0xffffffff = x mod 2^32`, `x >> 32 = x / 2^32` on `Int64`), 14 s; about half an hour of work for an agent, most of it on the shape of the goals (see the risks).
- The access to a row `C[i]` or `T[j]` of a 2D global: a lemma `row_split` (from VST's `split3_data_at_Tarray` and `data_at_singleton_array_eq`) and `row_addr` (`field_address0` is `offset_val (48 i)`), 32 lines, 1.8 s.
- `ExpTable.v` and the present `ExpConsts.v` of the constants agent (`exp100/rocq/`, written for the switch `native`) also compile in the switch `vst`: 0.9 s and 4.9 s.

2 The objects

A number is a list `a` of 6 integers, each in $[0, 2^{32})$, least significant first, held in `uint64_t` cells; `L(a)` is its value, the `valZ` of `ExpConsts.v` (the same shape as `Words.valZ` of `code/APaul/vst`, with limbs of 32 bits instead of 64). In VST it is

```
num sh a p := data_at sh (tarray tulong 6) (map Vlong (map Int64.repr a)) p
```

and every statement below asks `Zlength a = 6` and `Forall limb a` of its inputs and gives the same of its outputs. Inputs read only take a `readable_share`, outputs a `writable_share`; input and output are separate `SEP` items, so the statements do not cover a call with the same array in both places. No call in `exp100.c` does that (checked by reading every call).

The constants are seen by the functions that read them as

```
consts gv := data_at Ers (tarray (tarray tulong 6) 64) (map vwords T) (gv _T) *
             data_at Ers (tarray (tarray tulong 6) 17) (map vwords C) (gv _C) *
             num Ers LN2 (gv _LN2) * num Ers RMAX (gv _RMAX) *
             data_at Ers tulong (Vlong (Int64.repr INV)) (gv _INV)
```

with `T`, `C`, `LN2`, `RMAX`, `INV` the lists of `exp100/rocq/ExpTable.v`. `ExpTable.v` is written by a script from `exp100_table.h`, so the script is not trusted: one lemma per constant, `gvar_init v_T = concat`

(map (map (fun z => Init_int64 (Int64.repr z))) T) and so on, by **reflexivity**, ties these lists to the data the C file is compiled with. `exp100.c` has no `main`, so (as in `Verif_htr3.v`) the proof is a `semax_func` and `consts gv` is in the precondition: it is what `main_pre` gives for these read-only globals in any program that links `exp100.c`.

3 The functions

function	statement (pre $\Rightarrow$ post), loops and their invariants	effort
<code>num_zero(a)</code>	<code>data_at_ $\Rightarrow$ num a [0;...;0]</code> . One counted loop: <code>cells < i</code> are 0, the others <code>Vundef</code> .	done (13 lines)
<code>num_copy(a,b)</code>	$\Rightarrow a' = b$. Loop: <code>a'</code> is <code>sublist 0 i b ++</code> the rest.	30 lines, guessed
<code>num_add(a,b)</code>	$L(a) + L(b) < 2^{192} \Rightarrow L(a') = L(a) + L(b)$. Loop: <code>a' = r ++ sublist i6a</code> , $L(r) + c2^{32i} = L(a_{[0,i]}) + L(b_{[0,i]})$, $c \in \{0,1\}$. Each step: $c + a_i + b_i < 2^{33}$.	done (95 lines)
<code>num_sub(a,b)</code>	$L(a) \geq L(b) \Rightarrow L(a') = L(a) - L(b)$. Loop: $L(r) - c2^{32i} = L(a_{[0,i]}) - L(b_{[0,i]})$, $c \in \{0,1\}$; the <code>if</code> on <code>a[i] >= t</code> is a <code>forward_if</code> .	100 lines, guessed
<code>num_lt(a,b)</code>	returns 1 iff $L(a) < L(b)$. Loop from $i = 5$ down to -1 with two <code>returns</code> inside: <code>forward_loop</code> with $a_k = b_k$ for $k > i$; one list lemma: comparing from the top limb is comparing the values.	100 lines, guessed
<code>num_mul_small(p,a,w)</code>	<code>p</code> has 7 cells, $w < 2^{32} \Rightarrow L(p') = L(a)w$ (7 limbs). Loop: $L(p'_{[0,i]}) + c2^{32i} = L(a_{[0,i]})w$, $c < 2^{32}$; step: $a_i w + c \leq 2^{64} - 2^{32}$.	80 lines, guessed
<code>num_mulshr(r,a,b)</code>	$L(a)L(b) < 2^{352} \Rightarrow L(r') = \lfloor L(a)L(b)/2^{160} \rfloor$. Local <code>p[12]</code> . Three loops: zero <code>p</code> ; outer i : $L(p) = L(a_{[0,i]})L(b)$, $p_k = 0$ for $k \geq i + 6$; inner j : $L(p) + c2^{32(i+j)} = L(a_{[0,i]})L(b) + a_i L(b_{[0,j]})2^{32i}$, cells limbs, $c < 2^{32}$; step $a_i b_j + p + c \leq 2^{64} - 1$; last loop copies <code>p_[5,11]</code> , whose value is the quotient (it fits).	250 lines, guessed
<code>num_bitlen(a)</code>	returns <code>bitlen (L a)</code> with <code>bitlen v = if v = 0 then 0 else Z.log2 v + 1</code> . Two nested loops, one invariant for both: $b = \text{bitlen}(L(a) \bmod 2^{32i+k})$.	150 lines, guessed
<code>num_pow2(a,f)</code>	$0 \leq f < 192 \Rightarrow L(a') = 2^f$. Calls <code>num_zero</code> ; <code>f / 32</code> , <code>f % 32</code> are signed <code>int</code> operations (<code>Int.divs</code> , <code>Int.mods</code>), equal to <code>Z.div</code> , <code>Z.modulo</code> on $f \geq 0$.	50 lines, guessed
<code>num_low(a,b,f)</code>	$0 \leq f < 192 \Rightarrow L(a') = L(b) \bmod 2^f$. Loop: $a'_{[0,i]} =$ the limbs of $L(b) \bmod 2^f$ below i ; three cases on i against $f/32$.	100 lines, guessed
<code>num_scale(a,v,e)</code>	$v < 2^{53}$, $-2^{31} < e < 128 \Rightarrow L(a') = \text{scale } ve$ ($v2^e$ for $e \geq 0$, $\lfloor v/2^{-e} \rfloor$ otherwise). No loop; cases $e < 0$, $e > -64$. The C comment says $e \leq 138$: wrong, $e \geq 128$ writes <code>a[6]</code> (the one call has $e \leq 117$).	100 lines, guessed
<code>mul_ln2(q,n)</code>	$n < 2^{32}$, <code>consts gv $\Rightarrow L(q') = \lfloor n \cdot L(\text{LN2})/2^{32} \rfloor$</code> . Local <code>p[7]</code> , calls <code>num_mul_small</code> , copy loop.	60 lines, guessed
<code>guess_n(X)</code>	$L(X) < 2^{170}$, <code>consts gv $\Rightarrow$ returns $g = \lfloor L(X) \cdot \text{INV} / 2^{185} \rfloor$</code> , $g < 2^{17}$. The <code> </code> joins disjoint bits. Only the bound is used later: n is checked.	60 lines, guessed

<code>exp_core(xb,y,s)</code>	see section 4. Six local arrays, eleven calls, the Horner loop counting down from 15 to 0 with invariant $L(h) = H_{i+1}$.	500 lines, guessed
<code>exp_encl_bits(xb,M,s)</code>	$rc = 0 \Rightarrow$ the 3 words of M stand for $L(y)$ (base 2^{64}), $*s = h_N - 160$, and the enclosure of step 4. Loop over $i < 3$ ($NL / 2$ in the C).	120 lines, guessed
<code>maybe_hard_bits(xb)</code>	returns 0 $\Rightarrow$ not hard x (section 6). Straight-line code over 13 calls; the integer facts of section 5.	300 lines, guessed

The VST files only use the statements of the callees, so all bodies can be proved at the same time once the statements are written.

4 `exp_core` in integers (step 2)

In a pure Rocq file `Exp100Spec.v` (no VST, no reals), from the word `xb` and the lists of `ExpTable.v`:

```

sgn xb := xb / 2^63          be0 xb := (xb / 2^52) mod 2^11
mx xb  := xb mod 2^52
X xb   := if be0 xb = 0 then scale (mx xb) (1 - 915)
        else scale (mx xb + 2^52) (be0 xb - 915)      (* 915 = 1075 - 160 *)
q n    := n * L("LN2") / 2^32
H r 16 := L(C_16)          H r i := H r (i+1) * r / 2^160 + L(C_i)

```

```

core_rel xb y hN := exists n,
  be0 xb < 1033 /\ 0 <= n /\ n + 1 <= 2^17 /\
  q n <= X xb < q (n + 1) /\
  let Nu := if sgn xb = 1 then 2^17 - (n + 1) else 2^17 + n in
  let r := if sgn xb = 1 then q (n + 1) - X xb else X xb - q n in
  r < L(RMAX) /\
  y = L(T_(Nu mod 64)) * H r 0 / 2^160 /\
  hN = Nu / 64 - 2048 /\ 2^160 <= y < 2^162

```

(all divisions are `Z.div`). The statement of `exp_core`:

```

PRE  0 <= xb < 2^64, y: 6 writable cells, s: one writable tlong, consts gv
POST EX rc, if rc = 0 then EX ys hN, num Ews ys y * s |-> hN /\ core_rel xb (L ys)
hN
      else the cells are left as data_at_

```

The VST proof needs these pure lemmas, in `Exp100Spec.v`:

- $L(X) < 2^{170}$ when $be0 < 1033$ (so `guess_n` gives $g < 2^{17}$, and every `mul_ln2` is called with $n + 1 < 2^{32}$);
- from $q(n) \leq X < 2^{170}$ and $L(LN2) \geq 2^{185}$ (by computation): $n + 1 \leq 2^{17}$, so $131072 - (n + 1)$ does not wrap;
- $r < L(RMAX) < 2^{155}$, so by induction $2^{160} \leq H_i < 2^{161}$ (each `num_mulshr` and `num_add` of the loop has its fit condition);
- $2^{160} \leq L(T_j) < 2^{161}$ (computation), hence $2^{160} \leq y < 2^{162}$, which `maybe_hard_bits` needs for $y - 16$ and $y + 16$.

A copy of `core_rel` as a function (the C guess included) can be run by `vm_compute` on a few test words against the output of `test_exp100`: a cheap check that the statement says what the C does, before any proof.

5 The real numbers (step 4) and the decision (step 5)

Both are pure Rocq files, with `Flocq`, `Coquelicot` and `Interval`, built in the switch `vst` (all three are installed there; `ExpHard.v` is already built in `code/APaul/vst/link`).

The double. $x := \text{B2R } (\text{b64_of_bits } xb)$ (Flocq's IEEE754.Bits, not CompCert's copy `compCert.flocq`, which must not be mixed in). Lemma `x_bits`: for $\text{be0} < 2047$, $x = (-1)^{\text{sgn}} m 2^{e-1075}$ with (e, m) as in `X`; hence $X \leq |x| 2^{160} < X + 1$.

Step 4, `Exp100Encl.v`:

```
Theorem core_encl xb y hN : 0 <= xb < 2^64 -> core_rel xb y hN ->
  Rabs (exp x - IZR y * bpow (hN - 160)) <= 16 * bpow (hN - 160).
```

One lemma per line of the budget of the README, then their sum: the reduced argument ($|ru - r^*| \leq 2u$, from `LN2_ok` and $n < 2^{17}$), the Horner floors ($e_i \leq e_{i+1}r + 2$), the Taylor remainder (`taylor_rem_le` of `ExpConsts.v`), the table (`T_ok`), the last floor. The budget leaves $3.18 u$ of slack, so the bounds can be loose.

Step 5, `Exp100Hard.v`, with nothing of `C` or of the constants:

```
Theorem decide_sound (x : R) (y s : Z) :
  Rabs (exp x - IZR y * bpow s) <= 16 * bpow s ->
  bitlen (y - 16) = bitlen y -> bitlen (y + 16) = bitlen y ->
  let e := s + bitlen y - 1 in let f := Z.max e (-1022) - 53 - s in
  0 <= f - 43 ->
  2 ^ (f - 43) + 16 < Z.min (y mod 2 ^ f) (2 ^ f - y mod 2 ^ f) ->
  ~ ExpHard.hard x.
```

Proof: $\exp(x)/2^s$ is in $[y - 16, y + 16] \subset [2^{b-1}, 2^b]$, so $\text{binade } (\exp x) = e$ and $\text{vof } x = 2^{f+s}$; every z in that interval is at distance at least $d - 16 > 2^{f-43}$ from the multiples of 2^f , so $\exp(x)/v$ is at distance more than $2^{-43} = \text{bp } (-m)$ from every integer.

6 The final theorem (step 6)

The statement of `maybe_hard_bits` is the theorem; it holds by `body_maybe_hard_bits` (which calls `core_encl` and `decide_sound` on the integers given by the statements of `exp_core`, `num_bitlen`, `num_low`, `num_pow2`, `num_sub`, `num_lt`):

```
Definition maybe_hard_bits_spec :=
  DECLARE _maybe_hard_bits
  WITH gv : globals, xb : Z
  PRE [ tulong ]
    PROP (0 <= xb < 2 ^ 64) PARAMS (Vlong (Int64.repr xb)) GLOBALS (gv)
    SEP (consts gv)
  POST [ tint ]
    EX rc : Z,
    PROP (rc = 0 -> ~ ExpHard.hard (B2R _ _ (b64_of_bits xb)))
    RETURN (Vint (Int.repr rc)) SEP (consts gv).
```

```
Lemma exp100_funcs_correct :
  semax_func Vprog Gprog (Genv.globalenv prog)
    [(_num_zero, Internal f_num_zero); ...; (_maybe_hard_bits, Internal
f_maybe_hard_bits)]
  Gprog.
```

over the 16 functions (all but `exp_encl` and `maybe_hard`), as `htr3_funcs_correct` of `Verif_htr3.v`, with the `gvar_init` lemmas of section 2 and `Print Assumptions` showing only VST's standard axioms. Not proved: the union wrappers, the generator `gen_table.c`, the tests, and the use of the result by the search (x must be one of the inputs the tables of `code/exptablekl` cover; outside $(-745.14, 709.79)$ the filter may answer 1 for every x , which is sound).

7 Risks

- **Shapes of goals.** Most of the time on `num_add` went on syntax, not mathematics: `Znth` with `Inhabitant_val` against `Vundef`, a hypothesis `Zlength r = i` that `Intros` substitutes, `NL` a definition that `list_solve` does not unfold, `entailer!` clearing a hypothesis. T0 should put the fixes in `Common.v` (`Znth_vwords`, `upd_vwords`, the word lemmas, `NL` as a notation or unfolded once) so the other tasks do not meet them again.
- **Signed and unsigned.** `be`, `f`, `e`, `q`, `b` are `int`; `hN`, `e`, `ve`, `f` of `maybe_hard_bits` are `int64_t`; the rest `uint64_t`. `forward` asks for no signed overflow (all values are below 2^{12} in size) and for shifts below 64 (`v >> (-e)` is guarded by `e > -64`). Casts: `(int)` of an 11-bit field, `(int64_t)(Nu >> 6)`, `(int) f. 131072 - (n + 1)` is unsigned: it needs $n + 1 \leq 2^{17}$ (section 4).
- **Aliasing.** The statements take inputs and output as separate `SEP` items; this is right for every call in the file, and would not cover `num_add(a, a)`.
- **Arrays of arrays.** `C[i]` and `T[j]` are rows of 2D read-only globals; `row_split / row_addr` (measured, 32 lines) give the row as a `num` at `offset_val (48 i)` and put it back after the call.
- **Size.** `exp_core` has six local arrays, the five constants and two outputs in its `SEP`, and eleven calls; `entailer!` may be slow on it (minutes per step, guessed). If so, cut the body proof with `sema_seq` lemmas or hide the constants in one opaque `consts gv`.
- **Loops not of the simple form.** `num_lt` and the Horner loop count down; `num_lt` returns from inside the loop. `forward_for_simple_bound` does not apply; use `forward_loop` or `forward_for` with a general invariant. Loop bounds written `2 * NL` and `NL / 2` reach `Clight` as `Ebinop` of constants; `forward_for_simple_bound` may need them as a literal (guessed, not tried).
- **Two switches.** The constants agent works in `native` (Rocq 9.1); the VST proof needs its files in `vst` (Rocq 9.0). Today they compile in both (measured); the `Makefile` of `code/APaul/vst/exp100` should build them there from `../rocq` (as `code/APaul/vst/link` does), so any break shows.
- **Names.** `ExpConsts.v` defines `valZ`, `limb`, `num`, `NL : nat`; `code/APaul/vst/Words.v` has another `valZ` (limbs of 64 bits). Use qualified names and do not import both.

8 Changes to the C that would help

None is needed. In order of use:

1. Fix the comment of `num_scale`: $e \leq 32(6 - 2) - 1 = 127$, not 138.
2. `num_lt` as a counted upward loop with no `return` inside (`r = 1` if `a[i] < b[i]`, `r = 0` if `a[i] > b[i]`, for $i = 0..5$): `forward_for_simple_bound` with invariant $r = \left[L(a_{[0,i]}) < L(b_{[0,i]}) \right]$.
3. Write `2 * NL`, `NL / 2`, `P / LIMB` as macros of literals (12, 3, 5).
4. `f` of `num_pow2`, `num_low` unsigned, so `/` and `%` are `Int.divu`, `Int.modu`.

9 The tasks

Each task is one or two files with fixed statements (written in T0 with `Admitted`); the agent replaces the `Admitted` and may use the statements of the others, not their proofs. Files in `code/APaul/vst/exp100/`, built in the switch `vst`; the probes of section 1 are there, a start for T0 and T1.

task	file	proves	needs	size (guessed)
T0	<code>Makefile</code> , <code>Common.v</code> , <code>Spec.v</code> , <code>Exp100Spec.v</code>	setup:- <code>clightgen</code> , <code>CompSpecs</code> , <code>num</code> , <code>consts</code> , <code>row lem-</code> <code>mas</code> , <code>word</code> <code>lem-</code>		1 day

		mas, the 16 state- ments, core_rel, H, bitlen, scale, the gvar_init lem- mas; stubs of every file; the vm_compute check of core_rel	
T1	Verif_simple.v	num_z100, num_copy, num_add, num_sub, num_mul_small	300 lines
T2	Verif_bits.v	num_l10 num_bitlen, num_pow2, num_low, num_scale	500 lines
T3	Verif_mulshr.v	num_m10shr	250 lines
T4	Verif_reduce.v	mul_l10, guess_n	120 lines
T5	Verif_core.v, lemmas in Exp100Spec.v	exp_c10e, with the bounds of section 4	500 lines
T6	Verif_top.v	exp_e10, statements of T8, T9 maybe_hard_bits	400 lines
T7	Exp100Bits.v	x_bit10 $X \leq$ $ x 2^{160} <$ $X +$ 1 (Flocq)	150 lines
T8	Exp100Encl.v	core_e10, constants (step 4)	600 lines
T9	Exp100Hard.v	decide_exp100.v only (step 5)	200 lines
T10	Exp100Final.v	semax10func over	50 lines

		the 16 func- tions, Print Assumptions , the README	
--	--	---	--

Order:

1. T0, by one agent or by hand; nothing else starts before it.
2. Then T1 to T9 in parallel: the VST tasks use only statements, T9 needs nothing of T0 but **bitlen**, T8 needs the constants of the other agent (their statements are enough to start). The largest are T5 and T8.
3. Then T10.

Total, guessed: about 3200 lines of proof, against about 2900 lines in all for `code/APaul/vst` (search of `htr3.c`), of which the real-number part (T7, T8, T9) is a quarter.